

Kryptographie

(Lösungsvorschläge zu den Übungen)

Henning Pagnia

DHBW Mannheim

Herbst 2025



Aufgabe 1 (Substitution-Chiffre)

Versuchen Sie, den folgenden Chiffretext zu entschlüsseln:

kudfgnextrirefpuyhrffryhatvfgqnfehr
pxtengqreqvtvgnyratrfryyfpungskud

Lösungsvorschlag

Hier ist wieder unser Chifftrat:

kudfgnextrirefpuyhrffryhatvfgqnfehrpxtengqreqvtvgnyratrfryyfpunsgkud

Das Knacken einer Substitution-Chiffre von Hand kann recht aufwändig sein. Daher testet man am besten zuerst, ob es sich um eine Verschiebe-Chiffre handelt, die ja Spezialfälle von Substitution-Chiffren sind. Ein Brute-Force-Angriff benötigt dann max. 25 Versuche. Noch einfacher ist eine Frequenzanalyse: Die häufigsten Buchstaben sind hier **r** (9-mal) und **f** (8-mal). Wenn man davon ausgeht, dass der Text ein deutscher Text ist, dann ist es durchaus wahrscheinlich, dass das **E** in ein **r** oder **f** überführt wurde, da **E** der häufigste Buchstabe in der deutschen Sprache ist.

Wenn man diese Vermutung prüft, dann wird man als **Summanden 13** zunächst probieren – und feststellen, dass es geht! Der Klartext lautet:

XHQSTARKEVERSCHLUESSELUNGISTDAS
RUECKGRATDERDIGITALENGESELLSCHAFTXHQ

Die XHQs wurden hier als Nachrichtenanfang und -ende verwendet; dies ist zunächst verwirrend, jedoch eigentlich eine weitere Schwachstelle (wieso?).

Anmerkung: Im vorliegenden Text trat das **E** neunmal auf; bei den 67 Buchstaben insgesamt ergibt sich eine Häufigkeit von **13,4%**. Für längere deutsche Texte wird die mittlere Häufigkeit für ein **E** mit ca. **17%** angegeben. Wenn man eine Tabelle der Buchstabenhäufigkeiten aus verschiedenen Sprachen verwendet, lässt sich auf diese Weise sogar die Sprache ermitteln.

Aufgabe 2 (Schablonen-Kryptogram)

Sender und Empfänger verwenden identische Schablonen (diese dienen als Schlüssel). Jede Schablone hat n Zeilen mit jeweils m Spalten für Felder mit jeweils einem Buchstaben. Nun werden an mehreren zufällig ausgewählten Feldern (ca. ein Drittel) Löcher ausgestanzt (1) und in die Löcher in der korrekten Reihenfolge die Buchstaben der Nachricht eingetragen (2). Die verbleibenden Felder werden mit den Buchstaben eines beliebigen Textes in zufällig permutierter Reihenfolge aufgefüllt (3). (Dies erhält die Buchstabenhäufigkeiten der verwendeten Sprache.)

Nun kann die Nachricht übermittelt werden (4), und der Empfänger kann mit der richtigen Schablone die Nachricht leicht wiederherstellen (5).

- (a) Probieren Sie es mit einer 8x6 Schablone aus!
- (b) Wie groß ist der ungefähre Aufwand für einen Brute-Force-Angriff?
- (c) Entwerfen Sie eine für einen Computer geeignete Implementierung!

Lösungsvorschlag

- (a) HkAcPP tebmYo uBIhfo rRtseT eHvDAd laeyYT OaYsmr he0coU
(Großbuchstaben sind Teil der Nachricht, Kleinbuchstaben Fülltext)
- (b) Wir haben $m \cdot n$ Felder und jedes kann gelocht sein oder nicht:
die Anzahl aller möglichen Schablonen ist $2^{m \cdot n}$
Hier sind das $2^{48} \approx 10^{15}$, also recht viele ...
Aber nicht alle Schablonen sind wirklich sinnvoll für die Verschlüsselung.
Grobe Abschätzung:
Eine Schablone hat pro Zeile m Spalten. In jede Spalte lassen sich sinnvollerweise durchschnittlich $m/3$ Löcher stanzen.
Es gibt $\binom{m}{m/3}$ Möglichkeiten diese zu verteilen.
Bei n Zeilen ergeben sich demnach ca. $\left(\binom{m}{m/3}\right)^n$ sinnvolle Schablonen.
Für die 8x6 Schablonen sind das dann nur noch ca. $\left(\binom{6}{2}\right)^8 \approx 2,6 \cdot 10^9$
- (c) Der Schlüssel kann als Vektor implementiert werden, wobei dessen Elemente die Löcher in der Schablone beschreiben. Aus einer Textdatei wird ein passender Ausschnitt ausgewählt und die Buchstaben werden zufällig permutiert. Anschließend kann der Chiffretext generiert werden.

Aufgabe 3 (Koinzidenzindex)

Schreiben Sie ein Programm, das zu einem Text die Buchstabenhäufigkeiten bestimmt und den Koinzidenzindex $\mathfrak{K}$ berechnet.

Lösungsvorschlag

```

class TextAnalyzer {
    static String alphabet = "abcdefghijklmnopqrstuvwxyz";

    public static void main(String[] arg){
        String s = "dieentscheidendeschwachstelleeinerverschiebchiffrebestehtdarindassidentischeklartextzeichen" +
            "immerinidentischechiffretextzeichenabgebildetwerdenausdiesemgrundhatdergeheimtextdiegleichen" +
            "statisticheneigenschaftenwiederklartextnunkannbestimmtwerdenwiehaeufigjederbuchstabeim" +
            "geheimtextaufauchdiehaeufigkeitsverteilungwirdderjenigenaehnelnwelchediezielspracheaufweist" +
            "sokannmandiezeichenidentifizierenundsomitentschluesseleindierelativhaeufigauftreten";

        int l = s.length();
        int nettolen = 0;
        char zeichen;
        int[] ctr = new int[alphabet.length()];
        for (int i=0; i<l; i++){
            zeichen = s.charAt(i);
            try{
                ctr[letterCode(zeichen)]++;
                nettolen++; // Textlaenge ohne andere Zeichen
            } catch(Exception e){}; // Leerzeichen u.ae. ignorieren
        }

        System.out.println("s=[" + s + "]");
        System.out.println("Laenge von s = " + l);
        for (int j = 0; j<alphabet.length(); j++){
            zeichen = alphabet.charAt(j);
            System.out.println(zeichen + ": " + ctr[j]);
        }

        double kIndex = 0;
        for (int i=0; i<alphabet.length(); i++){
            kIndex += ctr[i]*ctr[i] ;
        }
        kIndex = kIndex / nettolen / nettolen;
    }
}

```



```
System.out.println(" Koinzidenzindex I = " + kIndex );
int n = l;
double h = (0.0377)*n / (kIndex*(n-1)-0.0385*n+0.0762);
System.out.println(" Friedman-Test ergibt (falls deutscher Text mit A...Z Buchstaben) KeyLength von ca. "+ h);
}
} // main

public static int letterCode(char c) {
    c = Character.toLowerCase(c); // konvertieren in Kleinbuchstaben
    for (int i=0; i<alphabet.length(); i++){
        if (c == Character.toLowerCase(alphabet.charAt(i)))
            return i;
    }
    throw new IllegalArgumentException("letter: " + c);
} // letterCode
} // class
```

Aufgabe 4 (Komprimierung)

Diskutieren Sie unter Bezug auf den Koinzidenzindex $\mathfrak{S}$, wie Verschlüsselung und Komprimierung aufeinander wirken.

Lösungsvorschlag

Verschlüsselung benötigt idealerweise eine Eingabe mit geringem Koinzidenzindex I (d. h. alle Zeichen aus dem Zeichenvorrat treten nahezu gleich häufig auf ($\Rightarrow$ kaum Redundanz)). Als Resultat einer guten Verschlüsselung entsteht ein Chiffretext mit kleinem $\mathfrak{S}$.

Komprimierung entfernt Redundanzen der Eingabe und reduziert somit $\mathfrak{S}$.
Eingaben mit nur wenigen Redundanzen lassen sich nur schlecht komprimieren.

Am besten ist es also, eine Eingabe zunächst zu komprimieren und anschließend zu verschlüsseln.

Aufgabe 5 (Affine Chiffren)

- (a) Wie lautet der Schlüssel k , der die historische Caesar-Chiffre beschreibt?
- (b) Gegeben sei $k = (9, 13)$ in $\mathbb{Z}_{26}$. Wie lautet der Klartext zum Chiffretext
n c c n f z
- (c) Entschlüsseln Sie den folgenden Chiffretext, der mit $k = (17, 1)$ generiert wurde:

ä u ß w ß

Dabei soll die folgende Kodierung gelten:

$$[A, B, \dots, Z, \ddot{A}, \ddot{O}, \ddot{U}, \ddot{B}] \rightarrow [0, 1, \dots, 25, 26, 27, 28, 29]$$

Wie groß ist hier der Schlüsselraum?

- (d) Zeigen Sie, dass eine Mehrfachverschlüsselung mit zwei Schlüsseln die Sicherheit nicht erhöht!

Lösungsvorschlag

(a) $k = (1, 3)$

(b) Aus $9 \cdot a^{-1} \equiv 1 \pmod{26}$ folgt $a^{-1} = 3$

der erste Chiffretextbuchstabe ist $c = 'n'$ also $c = 13$

$$\Rightarrow m = 3 \cdot (13 - 13) \pmod{26} = 0$$

$\Rightarrow m = A$ als erstes sowie T T A C K als restliche Klartextzeichen

(c) Aus $17 \cdot a^{-1} \equiv 1 \pmod{30}$ folgt

$a^{-1} = 23$ (man muss hier nur 13 und 23 probieren $\Rightarrow$ Wieso?)

$$c = (26, 20, 29, 22, 29) \Rightarrow m = (5, 17, 14, 3, 14) = F R O D O$$

$\varphi(30) = 8 \Rightarrow$ es gibt $8 \cdot 30$ verschiedene Schlüssel

$k = (1, 0)$ sollte man allerdings keinesfalls verwenden.

(d) Seien $k_1 = (a, b)$ und $k_2 = (x, y)$ und der Modulus sei n . Dann ist :

$$e_{k_1}(m) \equiv (a \cdot m + b) \pmod{n} = c_1 \text{ und } e_{k_2}(c_1) \equiv (x \cdot c_1 + y) \pmod{n} = c_2$$

$$\Rightarrow c_2 = (x \cdot [(a \cdot m + b) \pmod{n}] + y) \pmod{n}$$

$$\Rightarrow c_2 = (ax \cdot m + bx + y) \pmod{n}$$

Dies entspricht einer Einfachverschlüsselung mit $k_3 = (ax, bx + y)$

Aufgabe 6 (One-Time Pad)

Der durch korrekte Verschlüsselung mit einem One-Time-Pad erhaltene Chiffretext lautet:

xzttreef

Wie lautet der Klartext?

Lösungsvorschlag

Aufgrund der Voraussetzungen für das One-Time-Pad, kommt jeder Klartext mit 8 Buchstaben mit gleich hoher Wahrscheinlichkeit in Frage. Eine Lösung ist ohne Kenntnis des Schlüssels unmöglich ($\Rightarrow$ perfekte Sicherheit).

Aufgabe 7 (One-Time Pad – eine Anwendung)

In der Vorlesung wurde das One-Time Pad (OTP) als einziges beweisbar sicheres Verschlüsselungssystem vorgestellt. Die beiden Teilnehmer sind dabei im Besitz jeweils derselben Folge von Zufallsbits. Mit Hilfe von beispw. Micro-SD-Cards können solche langen Zufallsfolgen sehr leicht gespeichert und verteilt werden.

(a) Nehmen wir an, der Datenträger fasst 8 GiB. Wie lange reicht ein solcher Datenträger zum Verschlüsseln der Daten für

- ein Mobilfunkgespräch mit 9600 bps
- eine Videokonferenz mit 6 Mbps

Für wieviele E-Mails der Größe 4 KiB reicht solch ein OTP-Schlüssel?

- (b) Wie lange dauert die Erzeugung der Zufallsbits? ($\Rightarrow$ Recherche im Internet)
- (c) Welche Vor- und welche Nachteile hat OTP gegenüber beispw. AES?

Lösungsvorschlag

(a) ($8 \text{ GiB} = 2^{36} \text{ bit}$) Beim Mobilfunkgespräch (9600 bps) erreicht man eine Sprechdauer von ca. 1988 h (beinahe 83 Tage), bei einer Videokonferenz mit 6 Mbps eine Übertragungsdauer von etwa 190 Minuten. Es lassen sich 2 097 152 E-Mails damit verschlüsseln.

(b) Nicht trivial ist die Generierung einer wirklich zufälligen Folge von Nullen und Einsen. Es gibt einige Angebote im Web, zum Beispiel unter <http://www.id.uzh.ch/cl/zinfo/zinfo0022/kattipps/chaos.html>; hier wird eine Rate von 16 Mbps angegeben.

Die Erzeugungsdauer für 8 GiB ist hier $4295 \text{ sec} \approx 71,5 \text{ min}$. Ein weiterer Generator findet sich z. B. unter random.org.

(c) Die Ver- und Entschlüsselung erfolgt beim OTP mittels einfachem XOR; das ist schnell und kann auch problemlos von langsamer Hardware durchgeführt werden. Aufwändig ist jedoch meist die Verteilung der Pads, insbesondere dann, wenn man mit vielen verschiedenen Partnern kommuniziert. Dafür erhält man eine Verschlüsselung, die auch die Geheimdienste nicht knacken können.

Aufgabe 8 (Modifiziertes *One Time Pad*)

Ein Lauscher fängt den folgenden Chiffretext ab, der mit einem Verfahren ähnlich dem One-Time-Pad verschlüsselt wurde:

```
gpfazawkthpeazkiaokdbytzmxolfelvdquzdd  
ulfktlmgnqiaafhouzmymvpdzteplrbruvjiawsez  
fheuvqpgpqqpnqymvuglopuzgpqfojbryibqaaa  
aujlmzdihanpplqwdfnplrigvkjaelqodxuedmah  
mdjliautxmjwkjassiqokloofhxqewptytlrmujl  
ooooenfhupeqkizwohspqexvgqrbjziiafltnpil  
vlqqdjathicrlnkaplvgfotpmiiqyjliautxmjw  
hvbfbhyktwkjathicrlnlauawdquaafexbroilyeq  
ynmzlnfjmllvqouxaxjlmpllaeqswxddjiambjeq  
lzuoarevzphozuldmufofjukivflmjvulvmzxueo  
atmbqqatytsymevlmjppizqohuehoemgiphwgmxz  
qwlo
```

Wie lautet der Klartext?

Hinweise zum Verfahren:

Der Klartext enthält nur die Buchstaben 'A' (Code 0), 'B' (Code 1), ..., 'Z' (Code 25). Der Schlüssel hat nur eine begrenzte, feste Länge und wird zur Codierung endlos oft aneinandergereiht.

Jedes Chiffretextzeichen berechnet sich, indem zum Klartextzeichen-Code der Code des entsprechenden Schlüsselzeichens modulo 26 addiert wird.

(**Tipp:** in dem deutschen Text dreht es sich um "Kryptografie")

Lösungsvorschlag

Durch die begrenzte Schlüssellänge handelt es sich nicht um ein perfekt sicheres OTP sondern um eine Vigenère-Chiffre.

Zunächst muss die Schlüssellänge bestimmt werden. Hierzu sucht man gleiche Gruppen von Chiffretextzeichen und bestimmt deren Differenz d . (Kasiski-Test)

jliautxmj an Pos. 163 und Pos. 271 $\Rightarrow d_1 = 108$

jathicrln an Pos. 246 und Pos. 291 $\Rightarrow d_2 = 45$

iiäf an Pos. 51 und Pos. 231 $\Rightarrow d_3 = 180$

wkja an Pos. 172 und Pos. 289 $\Rightarrow d_4 = 117$

ggt(108, 45, 180, 117) = 9 (Vermutung für Schlüssellänge: 9 oder 3)

Wir überprüfen mittels des Friedman-Tests:

$I = 0,04598855612369126 \Rightarrow h = 4,989$ (als Näherung für Schlüssellänge)

Lösungsidee: Der Angriff geht prinzipiell natürlich immer mittels Brute-Force. Dazu müssen Sie hier jedoch $26^9 \approx 5,4 \cdot 10^{12}$ mögliche Entschlüsselungen erzeugen und prüfen – was recht aufwändig ist.

Schneller als Brute-Force ist eine Frequenzanalyse, die hier pro Spalte getrennt durchgeführt werden muss. Zur Schlüssellänge gibt es eine Vermutung (s.o.), anderenfalls könnte man auch einfach verschiedene Schlüssellängen durchprobieren. Allerdings wird dieses Vorgehen bei einem so kurzem Text nicht zwingend erfolgreich sein.

Besser funktioniert hier ein Known-Plain-Text-Angriff:

Versuchen Sie für ein möglichst langes Wort (idealerweise so lang oder länger als die Schlüssellänge), das mit hoher Wahrscheinlichkeit im Text vorkommt ('KRYPTOGRAPHIE', 'SCHLUESSEL', 'KLARTEXT', 'ALICE', 'EINE', etc.), den daraus resultierenden Schlüssel(teil) zu ermitteln und probieren Sie aus, ob Sie damit die restlichen Teile des Klartextes bestimmen können. Falls nicht, dann lassen Sie das geratene Wort sukzessive über den gesamten Text gleiten, denn es kann ja an einer beliebigen Position im Text auftreten ...

Falls Sie keinen Erfolg hatten, probieren Sie ein anderes Wort!

gpfazawkt dhbwmheim dieentsch	hpeazkiao dhbwmheim eidendesc	kdbytzmx dhbwmheim hwachstel	olfezlvdq dhbwmheim leeinerve
uzddulfkt dhbwmheim rschiebch	lmgnqiaf dhbwmheim iffrebest	houzmymp dhbwmheim ehtdarind	dzteplrbu dhbwmheim assidenti
vjiawsezf dhbwmheim scheklart	heuvqpgpq dhbwmheim extzeiche	qpniqymvu dhbwmheim nimmerini	gloupzgpq dhbwmheim dentische
fojbryibq dhbwmheim chiffrete	aaaaujlmz dhbwmheim xtzeichen	dihanpplq dhbwmheim abgebilde	wdfnplrig dhbwmheim twerdenau
vkjaelqod dhbwmheim sdiesemgr	xuedmahmd dhbwmheim undhatder	jliautxmj dhbwmheim geheimtex	wkjassiqo dhbwmheim tdiegleic
kloofhxqe dhbwmheim henstatis	wptytlrmu dhbwmheim tischenei	jloooooenf dhbwmheim genschaft	huxeqkizw dhbwmheim enwiederk
ohspqexvg dhbwmheim lartextnu	qrbjziiaf dhbwmheim nkannbest	ltnpilvlq dhbwmheim immtwerde	qdjathicr dhbwmheim nwiehaeuf

lnkaplvjg dhbwmheim igjederbu	fotpmiiqy dhbwmheim chstabeim	jliautxmj dhbwmheim geheimtex	whvbfhykt dhbwmheim tauftauch
wkjathicr dhbwmheim tdiehaeuf	lnlauawdq dhbwmheim igkeitsve	uafexbroi dhbwmheim rteilungw	lyezqynmz dhbwmheim irdderjen
lnfjmllvq dhbwmheim igenaehne	ouxaxjlp dhbwmheim lnwelched	llaeqswxd dhbwmheim iezielspr	djiambjeq dhbwmheim acheaufwe
lzuoarevz dhbwmheim istsokann	phozuldmu dhbwmheim mandiezei	fofjukivf dhbwmheim chenident	lmjvulvmz dhbwmheim ifizieren
xueoatmbq dhbwmheim undsomite	qatytsyme dhbwmheim ntschlues	vlmjppizq dhbwmheim selndiere	ohuehoemg dhbwmheim lativhaeu
iphwgmxxzq dhbwmheim figauftre	wlo dhb ten		

Aufgabe 9 (Zufallszahlen)

Stellen Sie für den Lehmer-PRNG $X_{n+1} = 31 \cdot X_n \bmod 61$ die erzeugten Pseudozufallszahlen als 2-Tupel in der Ebene dar (z. B. mit Excel o.ä.). Beurteilen Sie den Generator hinsichtlich der Qualität der erzeugten Zufallszahlen.

Lösungsvorschlag

Der PRNG ist minderwertig: Die Anzahl der erzeugten Werte ist viel zu klein und die erzeugten Punkte liegen alle auf zwei Geraden. Bei einem TRNG sollten sich keine regelmäßigen Strukturen bilden. Wenn ein Wert und die Parameter des PRNG bekannt sind, lassen sich alle weiteren Werte vorhersagen. Wenn eine ausreichend lange Folge von Zufallszahlen bekannt ist, lassen sich die Parameter berechnen.

Aufgabe 10 (Zufallszahlen)

Seien $p = 191$, $q = 167$, $s = 599$ die Parameter für einen BBS-PRNG.

- (a) Überprüfen Sie, ob die vorgegebenen Werte geeignet sind.
- (b) Erzeugen Sie eine 100 Bit lange pseudozufällige Bitfolge $z_1, \dots, z_{100}$.
- (c) Überprüfen Sie die Qualität mittels des Run-Tests!
- (d) Wieso müssen die Parameter zufällig gewählt werden?

Lösungsvorschlag

(a) $n = 31\,897$

Die Werte sind geeignet:

$$191 \bmod 4 = 3, 167 \bmod 4 = 3, \text{ggt}(599, 31\,897) = 1$$

(b) $x_0 = 599^2 \bmod 31\,897 = 7934$

$$x_1 = 7934^2 \bmod 31\,897 = 15\,575 \text{ (ungerade)} \Rightarrow z_1 = 1$$

$$x_2 = 15\,575^2 \bmod 31\,897 = 3940 \text{ (gerade)} \Rightarrow z_2 = 0 \quad \text{usw.}$$

$\Rightarrow 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1$

(c) Anzahl der Runs $k = 51$.

$$E(R) = (100 + 1)/2 = 50,5 \Rightarrow \text{optimal!}$$

Allerdings garantiert dies alleine noch nicht eine gute Qualität des PRNGs; es müssen unbedingt weitere Testmethoden angewandt werden!

(d) Es handelt sich um eine deterministische Funktion. Die Parameter dürfen einem Angreifer nicht bekannt sein, anderenfalls kann er die weitere Folge ggf. vorausberechnen.

Aufgabe 11 (ECB-Betriebsart)

In der Vorlesung wurde behauptet, dass die ECB-Betriebsart Bilder nicht sicher verschlüsselt, und es wurde ein Beispiel gezeigt.

In dieser Aufgabe sollen Sie selbst ein solches AES verschlüsseltes Bild herstellen.

Vorgehensweise:

- (1) Suchen Sie sich ein geeignetes Bild (z. B. Ihr Firmenlogo) und konvertieren Sie es in das PPM Format (z. B. mittels Photoshop, GIMP oder ffmpeg)
- (2) Entfernen Sie den Header (i.d.R. die ersten drei Zeilen, manchmal vier), achten Sie dabei darauf, die Kodierung nicht z. B. mit einem Texteditor zu zerstören. (Tipp: Nutzen Sie die Unix-Tools `head` und `tail`.)
- (3) Verwenden Sie `openssl`¹, um die resultierende Datei zu verschlüsseln (Parameter: `enc -aes-128-ecb -nosalt -pass pass:PASSWORD`)
- (4) Fügen Sie bei der Ausgabedatei den Header wieder hinzu und öffnen Sie das so entstandene PPM-Bild mit einem Bildbetrachter

¹ Kommandozeilen-Tool; vgl. <https://www.heise.de/download/product/openssl-22987> bzw.

<https://www.heise.de/download/product/win32-openssl-47316>

Lösungsvorschlag

Das nachstehende Unix Shell-Script² leistet diese Aufgabe:

```
#!/bin/sh
# Usage: ./ecb_img <image file> <password>
# This is part of my blog about AES: https://medium.com/p/7616beaaade9
# Inspired by https://blog.filippo.io/the-ecb-penguin/

# Convert your image to .ppm e.g. with "ffmpeg -i infile.gif outfile.ppm" or Gimp or Photoshop
ffmpeg -i $1 $1.ppm

# extract header and body
head -n 3 $1.ppm > $1.ppmheader.txt
tail -n +4 $1.ppm > $1.ppmbody.bin

# encrypt with ecb and given password
openssl enc -aes-128-ecb -nosalt -pass pass:"$2" -in $1.ppmbody.bin -out $1.ppmbody.ecb.bin

# encrypt with cbc and given password
openssl enc -aes-128-cbc -nosalt -pass pass:"$2" -in $1.ppmbody.bin -out $1.ppmbody.cbc.bin

# reassemble image
cat $1.ppmheader.txt $1.ppmbody.ecb.bin > $1.ecb.ppm
cat $1.ppmheader.txt $1.ppmbody.cbc.bin > $1.cbc.ppm

# Clean up temp files
rm $1.ppmheader.txt
rm $1.ppmbody.bin $1.ppmbody.ecb.bin $1.ppmbody.cbc.bin
echo
echo "encrypted image to $1.ecb.ppm"
echo " ... and $1.cbc.ppm"
```

² leicht angepasst von <https://gist.github.com/patrickfav/13b2f7272eaf91e3a72d87ac427485cb1>

Aufgabe 12 (GCM-Betriebsart)

Der Galois-Counter-Mode (GCM) gilt zur Zeit als sehr empfehlenswerte Betriebsart für den AES.

- (a) Wieso?
- (b) Implementieren Sie die Verschlüsselung mittels Java.
Tipp: Lesen Sie hierzu

Security Best Practices: Symmetric Encryption with AES in Java and Android von P. Favre-Bull (2018)
<https://proandroiddev.com/security-best-practices-symmetric-encryption-with-aes-in-java-7616beaaade9>

Lösungsvorschlag

(a) Verschlüsselung plus Authentisierung; durch Parallelisierbarkeit performant

(b)

```
import java.security.*;
import javax.crypto.*;
import javax.crypto.spec.*;
import java.io.*;
import java.nio.*;
import java.util.*;

public class AESTest {
    public static void main(String[] args){
        try{
            String plainText = "Hello World";
            byte[] associatedData = null; // AAD (optional)
            // choose a random key for AES-128
            SecureRandom secureRandom = new SecureRandom(); // use crypt. strong PRNG
            byte[] key = new byte[16];
            secureRandom.nextBytes(key);
            SecretKey secretKey = new SecretKeySpec(key, "AES");
            // choose 12 Bit IV for GCM
            byte[] iv = new byte[12]; //NEVER REUSE THIS IV WITH SAME KEY
            secureRandom.nextBytes(iv);

            String cipher64 = encrypt(plainText, secretKey, iv, associatedData);
            System.out.println("Base64-ciphertext=" + cipher64);
            System.out.println();

            System.out.println("Plain text=" + decrypt(cipher64, secretKey, associatedData));

            Arrays.fill(key, (byte) 0); //overwrite the content of the key with zeros
        } catch(Exception e) {e.printStackTrace();}
    } // main
}
```

```

public static String encrypt (String plainText, SecretKey secretKey, byte[] iv, byte[] associatedData){
    String msg = "";
    try{
        // choose cipher
        final Cipher cipher = Cipher.getInstance(" AES/GCM/NoPadding");
        GCMParameterSpec parameterSpec = new GCMParameterSpec(128, iv); //128 bit auth tag length
        cipher.init(Cipher.ENCRYPT_MODE, secretKey, parameterSpec);
        if (associatedData != null) {
            cipher.updateAAD(associatedData);
        }
        // encrypt
        byte[] cipherText = cipher.doFinal(plainText.getBytes());

        ByteBuffer byteBuffer = ByteBuffer.allocate(4 + iv.length + cipherText.length);
        byteBuffer.putInt(iv.length);
        byteBuffer.put(iv);
        byteBuffer.put(cipherText);
        byte[] cipherMessage = byteBuffer.array();
        msg = Base64.getEncoder().encodeToString(cipherMessage);
    } catch(Exception e){e.printStackTrace();}
    return msg;
} // encode

public static String decrypt(String base64Msg, SecretKey secretKey, byte[] associatedData){
    byte[] plainText = null;

    try{
        byte[] msg= Base64.getDecoder().decode(base64Msg);
        ByteBuffer byteBuffer = ByteBuffer.allocate(msg.length);
        byteBuffer = ByteBuffer.wrap(msg);
        int ivLength = byteBuffer.getInt();
        if(ivLength < 12 || ivLength >= 16) { // check input parameter
            throw new IllegalArgumentException("invalid iv length");
        }
        byte[] iv = new byte[ivLength];
        byteBuffer.get(iv);
        byte[] cipherText = new byte[byteBuffer.remaining()];
        byteBuffer.get(cipherText);

        final Cipher cipher = Cipher.getInstance(" AES/GCM/NoPadding");
        cipher.init(Cipher.DECRYPT_MODE, secretKey, new GCMParameterSpec(128, iv));
    }
}

```



```
        if (associatedData != null) {  
            cipher.updateAAD(associatedData);  
        }  
        plainText= cipher.doFinal(cipherText);  
    } catch (Exception e){e.printStackTrace();}  
    return new String (plainText);  
} // decode  
} // class AESTest
```

Aufgabe 13 (Diffie-Hellman-Schlüsseltausch)

Führen Sie das DHKE-Verfahren von Hand durch:

p sei 17, s sei 7. Alice wählt $a = 5$, Bob wählt $b = 6$.

- (a) Zeigen Sie, dass die Parameter geeignet sind!
- (b) Welche Werte werden ausgetauscht, wie lautet der Schlüssel?
- (c) Verwenden Sie den Schlüssel, um mittels der Verschiebe-Chiffre die Nachricht $m = \text{NOTSECRET}$ auszutauschen.
- (d) Spielen Sie den Angreifer, und führen Sie einen Brute-Force Angriff auf den Schlüsseltausch durch, um an Alices und Bobs Schlüssel zu kommen. (Ein 'echter' Angreifer würde hier wohl die Verschiebechiffre angreifen!)
- (e) Wieso darf hier nicht $a = 1$ oder $b = 1$ gewählt werden?
Wieso darf hier nicht $a = 16$ oder $b = 16$ gewählt werden?

Lösungsvorschlag

- (a) $a = 5 < p - 1$, $b = 6 < p - 1$, p prim,
 $s < p - 1$ ist ein Erzeuger (überprüfen mit Variante 1: Brute-Force).
- (a) Global bekannt sind die Parameter $p = 17$ und $s = 7$
Alice wählt $a = 5$ und berechnet $\alpha = s^a \bmod p = 11$
Bob wählt $b = 6$ und berechnet $\beta = s^b \bmod p = 9$
Alice berechnet $K_{alice} = \beta^5 \bmod 17 = 8$ und
Bob berechnet $K_{bob} = \alpha^6 \bmod 17 = 8$
- (c) Verschiebe-Chiffre mit $k = 8 \Rightarrow c = \text{vwbamkzmb}$
- (d) Ein Angreifer versucht, durch Ausprobieren a aus α (oder b aus β) zu bestimmen:
Z. B. $(\exists a \in \{2, \dots, 15\} \mid 7^a \equiv 11 \bmod 17)$
und kann dann einfach K berechnen

Prinzipielle Schwäche des Verfahren:

Alice weiß nicht, ob sie den Schlüssel mit Bob oder mit einem Angreifer tauscht, der den beiden Partnern die Identität des jeweils anderen vorspielt (Middle-Person-Angriff).

Originalveröffentlichung:

W. Diffie, M. Hellman: New Directions in Cryptography, *IEEE Transactions on Information Theory*, Nov. 1976, pp. 644-654.

(e) Bei $a = 1$ ergibt sich $\alpha = s$ wobei s öffentlich bekannt ist.

Falls also $\alpha = s$, ergibt sich daher unmittelbar der Wert $a = 1$ und K kann nun leicht berechnet werden. (b analog)

Bei $a = p - 1$ ergibt sich immer $\alpha = 1$
(nach dem kleinen Fermatschen Satz: $s^{p-1} \equiv 1 \pmod{p}$, $\forall s$, falls p prim).

Falls also $\alpha = 1$, ergibt sich daher unmittelbar der Wert $a = p - 1$ und K kann nun leicht berechnet werden. (b analog)

Aufgabe 14 (Primzahlen)

Berechnen Sie alle Primzahlen zwischen 1 und 100 000!

Welches ist die 1000. Primzahl?

Lösungsvorschlag

1000. Primzahl = 7919. Java-Programm für Sieb des Eratosthenes:

```
import java.io.*;
import java.util.*;
public class Sieb {
    public static void main(String[] args) throws IOException {
        PrintWriter f = new PrintWriter (new FileWriter("Primzahlen.txt"));
        int max = 100000;
        ArrayList<Boolean> primzahlen = siebe(max);
        System.out.println("Die Primzahlen bis " + max + " sind:");
        for (int i = 0; i <= max; i++)
            if (primzahlen.get(i) == true) {
                System.out.println(i);
                f.println(i);
            }
        f.close();
    } // main

    public static ArrayList<Boolean> siebe(int max) {
        // erzeuge ArrayList fuer potenzielle Primzahlen => true
        ArrayList<Boolean> liste = new ArrayList<Boolean>();
        for (int i = 0; i <= max; i++)
            liste.add(i, true);
        liste.set(0, false); // 0 ist nicht prim
        liste.set(1, false); // 1 ist nicht prim
        // wir testen alle Zahlen beginnend ab 2 (2 ist prim)
        for (int i = 2; i <= max; i++)
            if (liste.get(i) == true) // i ist prim
                // streiche alle Vielfachen von i
                for (int j = i+i; j <= max; j += i)
                    liste.set(j, false);

        return liste;
    } // main
} // class
```

Aufgabe 15 (RSA)

Gegeben seien die beiden Primzahlen $p = 11$ und $q = 17$. Der Exponent sei $e = 107$.

- (a) Wie lauten $E(e, n)$ und $D(d, n)$?
- (b) Verschlüsseln Sie die Nachricht $m = 6$ mit $E(e, n)$ und entschlüsseln Sie das Ergebnis c mit $D(d, n)$ um wieder m zu erhalten.

Lösungsvorschlag

(a) $p = 11$ und $q = 17$, also gilt: $n = p \cdot q = 187$.

Gewählt wird $e = 107$ (relativ prim zu $(p - 1)(q - 1)$)

Für d gilt: $e \cdot d \bmod (p - 1)(q - 1) = 1$, d.h. $107 \cdot d \bmod 160 = 1$

Man findet $d = 3 \Rightarrow D = (3, 187)$ und $E = (107, 187)$

(b) Die Nachricht ist $m = 6$.

$$c = m^e \bmod n \Rightarrow c = 6^{107} \bmod 187 = 107$$

Zur Berechnung von 6^{107} überlegt man sich, wie sich 107 aus Zweierpotenzen zusammensetzen lässt:

$$107 = 64 + 32 + 8 + 2 + 1.$$

$$\text{Mithin ist } 6^{107} = 6^{64} \cdot 6^{32} \cdot 6^8 \cdot 6^2 \cdot 6.$$

(Modulo-Operator darf beliebig oft angewendet werden $\Rightarrow$ wieso?)

Man berechnet sich also Hilfswerte:

$$6 \bmod 187 = 6$$

$$6^2 \bmod 187 = 36$$

$$6^4 \bmod 187 = (36^2 \bmod 187) = 174 (= -13)$$

$$6^8 \bmod 187 = (-13^2 \bmod 187) = 169 (= -18)$$

$$6^{16} \bmod 187 = 137$$

$$6^{32} \bmod 187 = 69$$

$$6^{64} \bmod 187 = 86$$

Weil $e = 107 = 1 + 2 + 8 + 32 + 64$ ergibt sich:

$$6^{107} \bmod 187 = 6 \cdot 36 \cdot 169 \cdot 69 \cdot 86 \bmod 187 = 107.$$

Nun zum Entschlüsseln von $c = 107$.

$$m = c^d \bmod 187 = 107^3 \bmod 187 = 6.$$

Aufgabe 16 (RSA)

- (a) Zeigen Sie am Beispiel $3^5 \bmod 7$, dass das *Ägyptische Potenzieren* korrekt rechnet.
- (b) Implementieren Sie das *Ägyptische Potenzieren* mittels Java in einer Klasse `MyRSA`.
Verwenden Sie hierzu die Klasse `BigInteger` (s. Java-API), um mit beliebig großen Zahlen arbeiten zu können!
- (c) Implementieren Sie zusätzlich den Erweiterten Euklidischen Algorithmus in die Klasse `MyRSA`.

Lösungsvorschlag

(a) $3^5 \bmod 7 = 243 \bmod 7 = 5$

5	3	relev.
$5 \text{ div } 2$ $= 2$	$3^2 \bmod 7$ $= 2$	–
$2 \text{ div } 2$ $= 1$	$2^2 \bmod 7$ $= 4$	relev.

$\Rightarrow$ berechne $3 \cdot 4 \bmod 7 = 12 \bmod 7 = 5$

Was haben wir berechnet?

$\Rightarrow (3^2 \bmod 7)^2 \bmod 7 \cdot 3 \bmod 7 = (3^2)^2 \cdot 3 \bmod 7 = 243 \bmod 7$

(b)+(c)

```

import java.math.*;

public class MyRSA {
    BigInteger ZERO = new BigInteger("0");
    BigInteger ONE  = new BigInteger("1");
    BigInteger TWO  = new BigInteger("2");

    // Hilfsmethoden
    private boolean isONE(BigInteger x){
        return (x.compareTo(ONE) == 0);
    }

    private boolean isODD(BigInteger x){
        return (((x.mod(TWO)).compareTo(ONE)) == 0);
    }

    // Aegyptisches Potenzieren
    private BigInteger compute(BigInteger k, BigInteger n, BigInteger x){
        BigInteger y = ONE;
        if (isODD(k))
            y = x;
        // System.out.println("k = "+k + "\tx = "+x);
        while (!isONE(k)) { // != 1
            k = k.divide(TWO);
            x = (x.pow(2)).mod(n);
            // System.out.println("k = "+k + "\tx = "+x);
            if (isODD(k))
                y = (y.multiply(x)).mod(n);
        }
        return y;
    } // compute

    public BigInteger encrypt (BigInteger e, BigInteger n, BigInteger m){
        return compute(e,n,m);
    }

    public BigInteger decrypt (BigInteger d, BigInteger n, BigInteger c){
        return compute(d,n,c);
    }
}

```

```

// Erweiterter Euklidischer Algorithmus
public BigInteger computePrivKey(BigInteger p, BigInteger q, BigInteger e ){
    BigInteger[] d = new BigInteger[100];
    BigInteger[] x = new BigInteger[100];
    int i = 0;
    BigInteger r = ONE;
    BigInteger z = (p.subtract(ONE).multiply(q.subtract(ONE)));
    System.out.println("z = " + z);
    do {
        r = z.mod(e);
        x[i] = z.divide(e);
        System.out.println("r = " + r);
        if (isONE(r)) break;
        z = e;
        e = r;
        i++;
    } while (true);
    for (int k=0; k<=i; k++)
        System.out.println("x["+k+"] = " + x[k]);

    // Anwenden der Rekursionsformel: d = A[0]
    d[i+1]=ONE;
    d[i+2]=ZERO;
    for (int k=i; k>=0; k- ){
        d[k] = d[k+2].subtract(x[k].multiply(d[k+1]));
        System.out.println(k + ": " + d[k]);
    }
    return d[0];
} // computePrivKey

public static void main(String[] args){
    MyRSA cipher = new MyRSA();
    // Testfall Aegyptisches Potenzieren
    BigInteger e = new BigInteger("53");
    BigInteger d = new BigInteger("17");
    BigInteger n = new BigInteger("77");
    BigInteger m = new BigInteger("6");
    BigInteger c = cipher.encrypt(e, n, m);
    System.out.println("\t\tChiffretext = " + c);
    BigInteger m2 = cipher.decrypt(d, n, c);
    System.out.println("\t\tPlaintext = " + m2);
}

```

```
// Testfall Erweiterter Euklidischer Algorithmus
BigInteger p = new BigInteger("257");
BigInteger q = new BigInteger("1009");
BigInteger e2 = new BigInteger("3553");
cipher.computePrivKey(p,q,e2);
    } // main
} // class
```

Aufgabe 17 (Schlüsseltausch mit RSA)

Beschreiben Sie, wie ein Schlüsseltausch mittels RSA abläuft.
Lässt sich hierbei *Perfect Forward Secrecy* erreichen?

Lösungsvorschlag

Alice wählt einen zufälligen, geeigneten Kommunikationsschlüssel K für ein symmetrisches Verschlüsselungsverfahren.

Alice verschlüsselt K mit öffentlichen RSA-Schlüssel von Bob und sendet dies an Bob. Außer Bob kann das niemand entschlüsseln, daher ist der Schlüsseltausch sicher.

Jeder, der einen Schlüsseltausch mit Bob durchführen möchte, verwendet Bobs öffentlichen RSA-Schlüssel. Falls dieser irgendwann bekannt werden sollte, lassen sich nachträglich alle mit Bob ausgetauschten Schlüssel und damit auch alle ausgetauschten Nachrichten rekonstruieren $\Rightarrow$ KEINE PFS!

Es wäre notwendig, dass jeder Teilnehmer für jeden Schlüsseltausch ein frisches zufälliges Schlüsselpaar generiert und nach erfolgtem Schlüsseltausch dieses unmittelbar sicher löscht. Auch müsste das neue Zertifikat wieder sicher abgefragt werden. $\Rightarrow$ unrealistisch hoher Aufwand!

Aufgabe 18 (Primzahltest)

Die Programmiersprache Python verwendet (ab Version 3) beliebig lange ganze Werte als Integer Werte (nicht für Gleitkommazahlen). Eine sehr effiziente Implementierung macht dies möglich.³ Daher bietet es sich an, Rechnungen mit sehr großen Zahlen – wie sie in der Kryptographie gängig sind – mit Python durchzuführen.

Implementieren Sie den Miller-Rabin-Primzahltest in Python und bestimmen Sie zu einer Fehlerwahrscheinlichkeit $< (\frac{1}{4})^{100}$ die kleinste Primzahl größer als 2^{1234} .

³ vgl. Arpit Bhayani: How python implements super long integers? (January 2010)
<https://www.codementor.io/@arpitbhayani/how-python-implements-super-long-integers-12icwon5vk>

Lösungsvorschlag

```
from random import randrange

# Miller Rabin: probabilistischer Primzahltest
def isPrime_MRT(n):
    exponent = n - 1
    u = 0

    while exponent % 2 == 0: # teile exponent bis Ergebnis ungerade -> u
        exponent //= 2
        u += 1

    # teste kleinsten Exponenten mit Zufallszahl
    a = randrange(2, n - 1)
    x = pow(a, exponent, n) # Square and Multiply
    if x == 1 or x == n - 1: # Fermat und Quadratwurzeltest
        return True

    # teste die Quadrate des Exponenten mit Quadratwurzeltest
    while u > 1:
        x = pow(x, 2, n) # Square and Multiply
        if x == 1:
            return False
        if x == n - 1:
            return True
        u -= 1
    return False
```

```

def main():
    x = 2**1234 + 805    # zu testende ungerade Zahl >=5
    isPrime = True
    s = 100              # Sicherheitsparameter
    for i in range(1, s):
        if (isPrime_MRT(x) == False):
            print(f"{x} is NOT prime")
            isPrime = False
            break
    if (isPrime):
        print(f"{x} is probably prime \n(error prob. <= 10^{-{(2*s/3.322)/1}})")

if __name__ == '__main__':
    main()

```

Lösung: $p = 2^{1234} + 805$ mit Fehler-WS $\leq 10^{-60}$

Aufgabe 19 (Primzahlproduktzerlegung)

Schreiben Sie ein Python Programm, welches das Quadratische Zahlenfeldsieb implementiert.

Zerlegen Sie mit Ihrem Programm die folgende sehr große Zahl in ihre beiden Primfaktoren p und q :

1044388881413152506691752710716624382579964249047383780384233483283953907971557456848826811934997558340890106714439262837987573438
 1857936072632360878513652779459569765437099983403615901343837183144280700118559462263763188393977127456723346843445866174968079087
 0580370407128404874011860911446797778359802900668693897688178778594690563019026094059957945343282346930302669644305902501597239986
 7714215541693835559885291486318237914434496734087811872639496475100189041349008417061675093668333850551032972088269550769983616369
 411933015213796825837188091833656751221318492846368125502259983004123447848625956744921946171077660876865116077929890850172521438
 1533661300611852571778780643838706072597426562044796568650681455378807327738225019010570540518637502455683329100682377363798329228
 4527779401216796709835850690521381013583202448370799186156489522196238248039801706889984061598510143551993564543378505292143052178
 8268872785126417810027990840568395363972762173111095691900450377667859350990328055178583545209800902741285878085041798115310370248
 4820606710124789790621255277247696513954759284633571334579257506116071596345263678342906908309258847855128421654798693568469322538
 7468951564347041644471458189153699117097101912389873234163020901

Hinweis: Berechnen Sie die Quadratwurzel beliebig großer ganzer Zahlen unter Verwendung von `math.isqrt()`.

Lösungsvorschlag

```
import math

# Quadratisches Zahlenfeldsieb
def qsieb(n, interval):
    b = 0
    while True:
        a = math.isqrt(n+b*b)
        if a*a == n+b*b:
            print("Anzahl Iterationen b=",b)
            return a-b, a+b
        b += 1
        if b > interval:
            return -1,-1

def main():
    p,q = qsieb(131*139, 100)
    print("n=",p*q, " p =",p, " q =",q, " Suchintervall=",100)

if __name__ == '__main__':
    main()
```

Lösung: $p = 2^{2048} + 981$; $q = 2^{2048} + 1617$

Aufgabe 20 (Elliptische Kurven)

Implementieren Sie in Java eine Klasse, mit der sich für die elliptischen Kurve $y^2 \equiv x^3 + 2 \cdot x + 2 \pmod{17}$ mit dem primitiven Element $P = (5, 1)$ die nachfolgenden Gruppenelemente $2P, 3P, \dots$ berechnen lassen.

Hinweise:

Implementieren Sie hierzu jeweils eine Methode für Addition und Verdoppelung sowie zur Berechnung der Inversen. Zusätzlich benötigen Sie eine Klasse *Punkt* für die Gruppenelemente (denken Sie auch an das neutrale Element).

Lösungsvorschlag

```
public class EC1 {  
    // EC:  $y^2 = x^3 + ax + b \text{ MOD } p$   
    private static int a = 1;  
    private static int b = 6;  
    private static int p = 11;  
  
    public static void main(String[] args) {  
        EC1 ec = new EC1(a,b,p);  
        Punkt[] p = new Punkt[100];  
  
        p[1] = new Punkt(5,9);  
        System.out.println("Primitives Element P = " + p[1]);  
  
        p[2] = ec.verdoppele(p[1]);  
        System.out.println("2P = " + p[2]);  
  
        for ( int i=3; i<23; i++){  
            p[i] = ec.addiere(p[1], p[i-1]);  
            System.out.println(" "+i+"P = " + p[i]);  
        }  
    } // main  
  
    public EC1 ( int a, int b, int p){  
        this.a = a;  
        this.b = b;  
        this.p = p;  
    } // Konstruktor
```

```

public Punkt verdoppele (Punkt z) {
    if (z.equals(Punkt.NULL)) return Punkt.NULL;
// System.out.println("Inverse (" + 2 * z.y + ", " + p + ") = " + inverse(2 * z.y, p));
    if (inverse(2 * z.y, p) == 0) return Punkt.NULL;
    int s = mod(((3 * z.x * z.x + a) * inverse(2 * z.y, p)), p);
    System.out.println("s = " + s);
    int x3 = mod((s * s - 2 * z.x), p);
    int y3 = mod((s * (z.x - x3) - z.y), p);
    return new Punkt(x3, y3);
} // verdoppele

public Punkt addiere(Punkt z1, Punkt z2) {
    System.out.println(" addiere(" + z1 + "+" + z2 + ")");
    if (z1.equals(Punkt.NULL)) return z2;
    if (z2.equals(Punkt.NULL)) return z1;
    if (z1.equals(z2)) return verdoppele(z1);

// System.out.println("Inverse (" + (z2.x - z1.x) + ", " + p + ") = " + inverse(z2.x-z1.x,p ));
    if (inverse(z2.x - z1.x, p) == 0) return Punkt.NULL;
    int s = mod((z2.y - z1.y) * inverse(z2.x - z1.x, p), p);
    int x3 = mod((s * s - z1.x - z2.x), p);
    int y3 = mod((s * (z1.x - x3) - z1.y), p);
    return new Punkt(x3, y3);
} // addiere

```



```

// Erweiterter Euklidischer Algorithmus
public int inverse(int e, int p){
    int pz = p;
    int[] d = new int[100];
    int[] x = new int[100];
    int i = 0;
    int r = 1;
    if (e == 1) return 1;
    if (e == 0) return 0;
    while (e < 0) e += pz;
    do {
        r = p % e;
        x[i] = p / e;
        // System.out.println("r = " + r);
        if (r == 1) break;
        p = e;
        e = r;
        i++;
    } while (true);
    for (int k=0; k<=i; k++)
        // System.out.println("x["+k+"] = " + x[k]);

    // Anwenden der Rekursionsformel: d = A[0]
    d[i+1]=1;
    d[i+2]=0;
    for (int k=i; k>=0; k--){
        d[k] = d[k+2] - (x[k] * d[k+1]);
        // System.out.println("t" + k + ": " + d[k]);
    }
    int erg = d[0];
    while (erg < 0) erg += pz;
    // System.out.println(d[0] + " | " + erg);
    return erg;
} // inverse

private int mod(int a, int p){
    while (a < 0 ) a += p; // bei negativen Zahlen anpassen
    return ( a % p );
} // mod
} // class EC1

```

```

class Punkt {
    // definiere neutrales Element
    public static final Punkt NULL = new Punkt(0,0);
    public int x;
    public int y;
    public Punkt(int x, int y){
        this.x = x;
        this.y = y;
    }

    public boolean equals(Punkt z){
        return (this.x== z.x) && (this.y == z.y);
    }

    public String toString() {
        String s;
        if (this.equals(NULL) )
            s = "Neutrales Element";
        else
            s = "("+this.x+";" +this.y+")";
        return s;
    }
} // class Punkt

```

Ausgabe:

Primitives Element P = (5,1)

2P = (6,3)

3P = (10,6)

4P = (3,1)

5P = (9,16)

6P = (16,13)

7P = (0,6)

8P = (13,7)

9P = (7,6)

10P = (7,11)

11P = (13,10)

12P = (0,11)

13P = (16,4)

14P = (9,1)

15P = (3,16)

16P = (10,11)

17P = (6,14)

18P = (5,16)

19P = Neutrales Element

20P = (5,1)

21P = (6,3)

22P = (10,6)

Aufgabe 21 (ECDH)

Die verwendete elliptische Kurve sei $y^2 = x^3 + x + 6 \bmod 11$

Alice hat $d_a = 6$ gewählt und erhält von Bob den Punkt $PB = (5, 9)$

Welchen gemeinsamen Schlüssel berechnet Alice?

Wieso kann Bob denselben Schlüssel berechnen?

Lösungsvorschlag

Alice berechnet $T_{AB} = d_A \cdot PB = 6 \cdot PB$

Wir verwenden das Java-Programm aus der vorigen Aufgabe und erhalten
 $T_{AB} = (2, 7) \Rightarrow$ Der gemeinsame Schlüssel ist $K_{AB} = x_{AB} = 2$

Alice berechnet $T_{AB} = d_A \cdot PB = d_A \cdot (d_B \cdot PE)$

Bob berechnet $T_{AB} = d_B \cdot PA = d_B \cdot (d_A \cdot PE)$

Da die Skalarmultiplikation hier kommutativ ist, erhalten beide denselben Schlüssel!

Aufgabe 22 (Non-Repudiation)

Symmetrische Verfahren alleine lassen sich nicht zum nachträglichen Beweis einer Kommunikation verwenden. Wieso nicht?

Lösungsvorschlag

Jeder, der diesen Beweis überprüfen möchte, muss hierzu den Schlüssel kennen. Nun kann aber jeder, der den Schlüssel kennt (also Alice, Bob und alle Prüfer), eine entsprechend verschlüsselte Nachricht erzeugen ...

Aufgabe 23 (Kryptografische Hash-Verfahren)

Gegeben sei eine Nachricht m mit n Blöcken der Länge 256 Bit.

Eine einfache Hash-Funktion besteht in der bitweisen XOR-Verknüpfung jeweils aller Bits an der Position $i \in \{1, 2, \dots, 256\}$ aller Nachrichtenblöcke $m_1 \dots m_n$:

$$h_i = m_{1,i} \oplus m_{2,i} \oplus \dots m_{n,i}$$

$$h = h_1 \circ h_2 \circ \dots \circ h_{256}$$

($\circ$ ist hierbei die Verkettung also Hintereinanderreihung der Bits)

Erläutern Sie, weshalb diese Hashfunktion nicht kryptografisch sicher ist!

Lösungsvorschlag

Es gibt die folgenden Probleme mit dieser Hashfunktion:

Bei Änderung eines Bits der Nachricht ändert sich nur ein einziges Bit im Hashwert.

Die Hashfunktion ist nicht kollisionsresistent: Man erstellt eine beliebige Nachricht mit dem gewünschten Inhalt und fügt einen zusätzlichen Textblock an. Die Bits dieses angefügten Blocks setzt man nun so, dass der benötigte Hashwert **h** entsteht.

Aufgabe 24 (Kryptografische Hash-Verfahren)

Kryptografische Hash-Verfahren erzeugen für beliebig lange Eingabetexte Werte einer vorgegebenen Länge, z.B. erzeugt das MD5-Verfahren 128-Bit lang Hashwerte. Kollisionen entstehen, wenn zwei unterschiedliche Texte denselben Hash-Wert erzeugen. Das Erzeugen von Kollisionen soll möglichst schwierig sein.

- (a) Wieviele Eingabetexte muss man bei einem geeigneten Hash-Verfahren erzeugen, damit mit einer Wahrscheinlichkeit von mehr als 50% zwei beliebige Eingaben denselben Hashwert erzeugen: bei 8-Bit, 16-Bit, 32-Bit, 64-Bit und 128-Bit langen Hash-Werten? (Stichwort: *Geburtstagsangriff*)
- (b) Erzeugen Sie zwei Texte, die denselben 8-Bit Hashwert besitzen. In dem einen verkaufen Sie ein Auto für 100 Euro und in dem anderen für 5.000 Euro. Wie könnten die beiden Texte von einem Betrüger verwendet werden?

Verwenden Sie die folgende Java-Methode zum Erzeugen der 8-Bit Hashwerte sowie als ersten Entwurf den nachfolgenden Vertragstext:

```

import java.security.*;
import java.io.*;

public class MD1 {
    public static byte generateMD (String filename) {
        byte result = 0;
        try {
            MessageDigest md = MessageDigest.getInstance("MD5");
            byte[] buffer = new byte [16];
            int n = 0;

            FileInputStream fileIn = new FileInputStream(filename);
            while ((n = fileIn.read(buffer)) > -1) {
                md.update(buffer, 0, n);
            }
            buffer = md.digest();
            fileIn.close();

            // Hashwert auf 8 Bit verkuerzen :
            result = buffer[0];
            for (int i = 1; i < 16; i++){
                result ^= buffer[i]; // XOR
            }
        } catch (Exception e) {System.err.println(e);}
        return result;
    } // generateMD

    public static void main (String[] args) {
        String filename = "Autoverkauf2.txt";
        byte result = MD1.generateMD(filename);
        System.out.println("Hashwert = <" + String.format("%2x",result)+ ">");
    } //main
} // class MD1

```

Mannheim, den 20.2.2020

Kaufvertrag

=====

zwischen

Herrn XYZ

und

Frau ABC.

Frau ABC verkauft hiermit ihren VW Polo mit wirtschaftlichem Totalschaden,
Baujahr 1999, mit dem amtlichen Kennzeichen MA-ABC 1999

zum Preis von

100 Euro

an Herrn XYZ.

Der Vertrag wird digital unterschrieben und ist mit sofortiger Wirkung gültig.

Lösungsvorschlag

(a) Das Problem entspricht dem aus der Literatur bekannten Geburtstagsangriff (bzw. auch Geburtstagsproblem genannt):

Bei MD5 gibt es $F = 2^{128}$ unterschiedliche Hash-Werte. Die Wahrscheinlichkeit, dass mindestens ein Eingabetext (von insgesamt N) genau einen bestimmten Hashwert-Wert erzeugt, ist die Gegenwahrscheinlichkeit dazu, dass keiner dies tut.

$$Prob = 1 - \left(1 - \frac{1}{F}\right)^N$$

Also gilt für $Prob \geq 50\%$:

$$1 - \left(1 - \frac{1}{F}\right)^N \geq 0.5$$

$$\left(1 - \frac{1}{F}\right)^N \leq 0.5$$

$$N \cdot \log_2\left(1 - \frac{1}{F}\right) \leq \log_2(0.5)$$

$$N \geq \frac{1}{\log_2\left(\frac{F}{F-1}\right)}$$

Aus M Eingaben lassen sich $N = \frac{M(M-1)}{2}$ Paare bilden:

$\Rightarrow$ Aufwand für den Angriff: $M = 0.5 + \sqrt{2N + 0.25}$

Damit gilt:

	für Umkehrung	für Kollision
$F=2^8$	$N \geq 178$	$M \geq 20$
$F=2^{16}$	$N \geq 45\,426$	$M \geq 302$
$F=2^{32}$	$N \geq 2\,977\,044\,471$	$M \geq 77\,164$
$F=2^{64}$	$N \geq 10^{19}$	$M \geq 6 \cdot 10^9$
$F=2^{128}$	$N \geq 2 \cdot 10^{38}$	$M \geq 4 \cdot 10^{19}$

(b) Bereits bei nur 20 Texten haben wir demnach eine Kollisionswahrscheinlichkeit von ca. 50 %. Wenn wir sowohl den Originalvertrag als auch den gefälschten jeweils mehrfach leicht abwandeln (Satzzeichen, Leerzeichen, synonyme Wörter, ...) ohne den Sinn zu verändern, finden wir recht bald die gewünschte Kollision.

Der betrügerische Verkäufer tauscht nach der Unterschrift des Käufers den originalen Vertragstext gegen einen mit einem höheren Kaufpreis aber demselben Hashwert aus. Weil der Hashwert identisch ist, gilt die Unterschrift des Käufers auch für den gefälschten Vertragstext.

Aufgabe 25 (Fiat-Shamir-Protokoll)

Gegeben für Bob: $n = 143$, $v = 82$

Mallory versucht, sich als Bob auszugeben und wählt $y' = 3$

- (a) Welchen Wert x' muss Mallory an Alice senden, damit Alice ihn für Bob hält, wenn Alice $e = 1$ wählt?
- (b) Was passiert dann, falls Alice sich für $e = 0$ entscheiden sollte?

Lösungsvorschlag

Mallory berechnet $x' = (y')^2 \cdot v^{-1} \bmod n$

Die Inverse zu v finden wir mit dem Erweiterten Euklidischen Algorithmus:

$$82 \cdot v^{-1} \equiv 1 \bmod 143$$

und wir erhalten $v^{-1} = 75$

$$\text{also ist } x' = 3^2 \cdot 75 \equiv 103 \bmod 143$$

$\Rightarrow$ Bob sendet $x' = 103$ an Alice

(a) Alice wählt $e = 1$

$\Rightarrow$ Bob sendet $y' = 3$ an Alice.

Probe: Alice verifiziert korrekt $3^2 \equiv 103 \cdot 82 \equiv 9 \bmod 143$

(b) Alice wählt $e = 0$

$\Rightarrow$ Bob muss r an Alice senden

$$r' \equiv (y')^2 \cdot s^{-1} \bmod n \text{ mit } v \equiv s^2 \bmod n$$

$\Rightarrow$ Bob muss die Quadratwurzel aus v berechnen und kann das nicht!